ENIAC/IAS-Style State-Transition Protocols for Reliable AI Agent Execution

A White Paper on Strict Skill Engineering, Agent Control Kernels, and the "Today" Opportunity

Abstract

AI agents are becoming capable enough to perform multi-step coding, analysis, document, and business-process tasks, yet their reliability remains uneven. The central weakness is not merely model intelligence, but execution discipline: agents often drift from instructions, skip implicit assumptions, over-edit, fail to preserve state, or self-declare success without verifiable evidence.

This paper proposes **ENIAC/IAS-style state-transition protocols** as a practical control layer for AI agents. The core idea is to convert ordinary natural-language tasks and reusable agent skills into explicit execution protocols of the form:

```
input state → operation step → output state → verification gate → next state
```

Two execution modes are proposed. **ENIAC-mode** represents fixed, linear, no-branch procedures where the plan is effectively "wired" before execution. **IAS-mode** represents stored-program execution with explicit program counter, branching, flags, validation, and recovery logic. Together, these modes provide a conceptual and practical grammar for making AI agents more stable, auditable, and reusable.

The paper further argues that the opportunity is especially urgent "today" because three forces have converged: many programmers are underemployed or displaced by AI pressure, enterprises urgently need stable AI automation, and current agent tools already expose enough extension mechanisms—skills, hooks, subagents, project instructions, scripts, and tool calls—to implement a first generation of strict agent workflows. This creates a short but significant window for programmers to become **Agent Skill Engineers**: professionals who translate messy human/business tasks into strict, reusable, verifiable AI execution protocols.

---

1. Problem Statement: AI Agents Are Powerful but Not Yet Procedurally Stable

Modern AI coding agents can read codebases, edit files, run commands, generate pull requests, and automate multi-step developer workflows. They are no longer merely autocomplete systems. They are increasingly autonomous task executors. [S1]

However, autonomy introduces a new class of problems:

1. The agent may misunderstand the task boundary.
2. The agent may silently make assumptions.
3. The agent may skip planning or revise the plan during execution.
4. The agent may change unrelated files.

5. The agent may claim completion without adequate evidence.
6. The agent may pass through a failure state without stopping.
7. The agent may rely on self-audit rather than external validation.
8. The agent may produce useful output once, but fail to reproduce the same procedure reliably.

These are not only “prompting problems.” They are **execution-control problems**.

Traditional software engineering solved similar problems through concepts such as preconditions, postconditions, invariants, transactions, logs, test suites, rollback, schemas, interfaces, and state machines. AI agents need a comparable procedural discipline.

This paper argues that the next practical reliability jump for AI agents will come from treating agent work not as free-form conversation, but as **controlled state transition**.

---

2. Core Thesis

The core thesis is:

> AI agent skills should be engineered as verifiable state-transition protocols, not merely as reusable prompt instructions.

A skill should not merely say:

> “Review this code carefully and fix any bugs.”

Instead, it should define:

```
Input State:
- Source files provided.
- Error message provided.
- Runtime or database engine known.
- Scope of allowed edits specified.

Step:
- Inspect only.
- No code modification allowed.

Output State:
- Module inventory.
- Suspected failure regions.
- Missing information.
- Risk level.

Gate:
- Do not proceed to modification unless evidence exists.
```

This transforms the agent from a loose assistant into a bounded execution worker.

In short:

```
Normal Agent Skill:
instruction → action → answer

Strict Agent Skill:
input state → operation → output state → validator → next step
```

---

3. ENIAC-Mode: Fixed Wiring for Linear Procedures

ENIAC-mode is used for tasks that can be expressed as a fixed sequence of steps with little or no branching.

Examples include:

- formatting files,
- generating documentation from a fixed template,
- converting schema A to schema B,
- applying known lint rules,
- producing a standard audit report,
- scanning a workbook using a defined checklist,
- extracting fields into a known JSON schema.

The metaphor is that the plan is “wired” before execution. Once the plan is frozen, the agent should not invent new steps unless a failure gate explicitly allows escalation.

A typical ENIAC-mode protocol is:

```
P0 SPEC
→ P1 PLAN FREEZE
→ P2 EXECUTE TRACE
→ P3 AUDIT
```

Each step has one expected input state and one expected output state.

Example:

```
Step 1:
Input State:
- Excel workbook is available.
- Target worksheet name is known.

Operation:
- Inspect headers only.

Output State:
- Header list.
- Column index map.
- Missing required columns.

Validator:
- Every reported header must exist in the inspected worksheet.
```

ENIAC-mode is especially effective where repeatability matters more than creativity.

Its strength is simplicity:

```
No hidden branches.
No vague progress.
No silent plan mutation.
No uncontrolled scope expansion.
```

---

4. IAS-Mode: Stored-Program Execution for Branching Procedures

IAS-mode is used when the task may require conditional branching, loops, retries, or failure handling.

Examples include:

- debugging SQL errors,
- fixing failing tests,
- migrating code between frameworks,
- refactoring multi-file modules,
- diagnosing build failures,
- implementing a feature with test feedback,
- resolving ambiguous legacy behavior.

IAS-mode treats the skill as a stored program. The agent has an explicit current step, allowed operations, flags, and branching rules.

A simplified IAS-style protocol contains:

```
STATE:
- PC: current step
- FLAGS: success/failure/uncertain
- MEMORY: accumulated facts, files, test results, assumptions
- LOG: append-only execution trace

OPCODES:
- INSPECT
- LOCALIZE
- HYPOTHESIZE
- PATCH
- TEST
- VERIFY
- REPORT
- ASK_USER
- HALT

INVARIANTS:
- Do not patch before evidence.
- Do not proceed after failed validation.
- Do not alter unrelated files.
- Do not convert hypothesis into confirmed fact.
```

A typical IAS loop is:

```
fetch current step
→ decode operation
→ execute bounded action
→ validate output state
→ update flags
→ branch, continue, ask user, or halt
```

IAS-mode is the natural mode for real programming work, because debugging and refactoring are rarely purely linear.

5. From Prompt Skill to State Skill

Most current agent skills are still written as rich instructions. This is useful, but incomplete. A stricter skill requires a grammar.

A proposed **Strict Skill Schema** includes:

```
skill:
  name:
  purpose:
  scope:
  risk_level:

input_state:
  required:
  optional:
  forbidden_assumptions:

allowed_operations:
  - INSPECT
  - PATCH
  - TEST
  - REPORT

forbidden_actions:
  - rewrite unrelated modules
  - delete files without explicit permission
  - claim test success without command output

steps:
  - id:
    mode:
    input_required:
    operation:
    output_required:
    validator:
    failure_policy:

completion:
  done_when:
  audit_report:
  unresolved_risks:
```

This schema turns a skill from a prompt into an executable contract.

The result is not full determinism. LLM reasoning is still needed. But the reasoning is now contained inside controlled steps.

The guiding principle is:

> The LLM may reason freely inside a bounded operation, but it may not freely redefine the workflow.

---

6. Verification: The Agent Must Not Be Its Own Judge

A central rule of strict agent engineering is:

> The same agent that performs the work should not be the only authority deciding whether the work succeeded.

Self-audit is useful, but insufficient. It should be supplemented by external validators.

Validators can include:

Validator Type	Example
Schema validator	Output must match JSON/YAML schema
Text validator	Cited evidence must exist in source text
Diff validator	Only allowed files/lines changed
Test validator	Unit tests, lint, type checks, SQL parser
Command validator	Required command executed successfully
Human gate	User approval needed before risky step
Independent model review	Separate reviewer checks reasoning

This is where modern agent platforms become relevant. Current coding-agent ecosystems already expose mechanisms such as skills, project instructions, hooks, subagents, plugins, and tool integrations that can support this layered design. [S2]

The best architecture is therefore not prompt-only. It is:

```
LLM = semantic worker
Skill = state-transition contract
Kernel = execution controller
Validator = external enforcement layer
Audit log = evidence trail
```

---

7. The Execution Kernel

A true strict-agent system requires an execution kernel.

The kernel is not necessarily a large system. A minimum viable version can be a CLI wrapper:

```
strict-agent run task.yaml
```

The kernel performs the following:

1. Load the state contract.
2. Check required input state.
3. Ask clarification questions if required data is missing.
4. Freeze the plan.
5. Execute one step at a time.
6. Validate the output state.
7. Commit, rollback, retry, branch, ask user, or halt.
8. Write an audit report.

A simple folder structure may look like:

```
.strict-agent/
  skills/
    sql-debug.yaml
    vba-review.yaml
```

```
    excel-report-generation.yaml

  validators/
    schema-validator.js
    diff-validator.js
    evidence-validator.js
    test-runner.js

  runs/
    2026-06-04-001/
      input-state.json
      plan.yaml
      step-01-output.json
      step-01-validation.json
      final-audit.md
```

This architecture changes the status of the LLM. The LLM is no longer the whole system. It becomes a processor inside a procedural runtime.

---

8. The “Today” Factor: Why This Matters Now

The word “today” is critical.

This is not only a long-term research direction. It is an immediate socio-technical opportunity because three forces have converged.

8.1 Many Programmers Need a New Economic Role

AI coding tools are changing the labour market. The pressure appears especially strong for early-career and AI-exposed software roles. The result is not simply that programmers become obsolete. Rather, many programmers are being pushed away from ordinary implementation tasks and need a higher-leverage role. [S3]

Strict skill engineering offers that role.

Programmers already understand:

- state,
- tests,
- logs,
- preconditions,
- postconditions,
- rollback,
- schemas,
- version control,
- execution traces,
- failure handling.

These are precisely the concepts needed to turn unstable AI workflows into stable agent skills.

Thus, the displaced or underemployed programmer can become an **Agent Skill Engineer**.

8.2 Businesses Need Stable AI Tasks, Not Just AI Chat

Enterprises are rapidly adopting AI, but the critical bottleneck is trust and control. Developers already use or plan to use AI tools at high rates, yet trust in AI-generated output remains limited. [S4]

This creates a gap:

```
High AI usage
+ Low AI trust
= Demand for verification and control layers
```

Businesses do not merely need AI that can answer. They need AI that can perform repeatable tasks safely:

- generate reports,
- review code,
- validate invoices,
- migrate scripts,
- produce test cases,
- inspect contracts,
- update documentation,
- audit configurations,
- transform data,
- diagnose errors.

These tasks require skill stability.

8.3 Current Platforms Already Have the Building Blocks

The opportunity is immediate because the necessary building blocks already exist.

Modern tools already support:

- reusable skills,
- project instructions,
- hooks,
- shell commands,
- scripts,
- subagents,
- plugins,

- MCP-style external tools,
- CI integration,
- local and cloud execution.

However, these parts are not yet widely unified into a simple developer-facing state-transition protocol.

That gap is the opportunity.

8.4 The Flywheel

The “today” opportunity can be expressed as a flywheel:

```
Underemployed programmers
→ learn strict skill engineering
→ convert messy business tasks into AI workflows
→ businesses obtain more reliable AI automation
→ trust and usage increase
→ demand for more strict skills increases
→ more programmers become Agent Skill Engineers
→ more reusable skills and kernels are created
```

This is a self-boosting cycle.

It is not only a product opportunity. It is a labour-market conversion mechanism.

---

9. Business Implication: A New Reliability Layer for the Agent Economy

The market opportunity is not simply “another AI coding assistant.”

The more important product category is:

> Agent reliability infrastructure.

Possible products include:

1. **Strict Skill Compiler**
   Converts ordinary prompts or skill documents into ENIAC/IAS-style protocols.
2. **Agent Execution Kernel**
   Runs state contracts step by step with validation gates.
3. **Validator Library**
   Provides reusable validators for code, SQL, spreadsheets, documents, schemas, APIs, and regulated workflows.
4. **Strict Skill Registry**
   Marketplace of reusable, audited agent skills.
5. **Agent CI System**
   Runs agent tasks in continuous integration with trace logs and rollback.
6. **Enterprise Agent Governance Layer**
   Monitors agent actions, cost, permissions, risks, and evidence.

The best commercial framing is not “prompt engineering.”

It is:

```
Deterministic control layer for AI agents.
State-verified workflow kernel.
Agent reliability compiler.
Agent governance and audit runtime.
```

---

10. Example: SQL Debugging Skill

A normal user request may be:

```
Why does this Oracle SQL give ORA-00907?
```

A strict IAS-mode skill converts it into:

```
skill: oracle_sql_debug
mode: IAS

input_state:
  required:
    - sql_text
    - error_message
  optional:
    - error_position
    - oracle_version
    - generated_sql_source

forbidden_assumptions:
  - do_not_assume_schema
  - do_not_rewrite_business_logic

opcodes:
  - INSPECT
  - LOCALIZE
  - HYPOTHESIZE
  - PATCH
  - VERIFY
  - REPORT
  - HALT

steps:
  - id: inspect_input
    operation: INSPECT
    output_required:
      - dialect
      - error_type
      - missing_inputs

  - id: localize_fault
    operation: LOCALIZE
    output_required:
      - suspicious_fragment
      - evidence
      - confidence

  - id: propose_minimal_patch
    operation: PATCH
    precondition:
      - suspicious_fragment exists
      - evidence exists
    output_required:
```

```
      - original_fragment
      - fixed_fragment
      - explanation

  - id: report
    operation: REPORT
    output_required:
      - confirmed_findings
      - hypotheses
      - remaining_uncertainties
```

This structure prevents common agent failure modes:

- It cannot patch before evidence.
- It cannot claim certainty without a cited fragment.
- It cannot rewrite the whole query unless allowed.
- It must separate confirmed bugs from hypotheses.
- It must halt or ask the user if required input is missing.

---

11. Example: VBA Workbook Review Skill

A normal request:

```
Review this VBA and tell me where to change it.
```

A strict ENIAC/IAS hybrid skill:

```
skill: vba_review
mode: IAS

input_state:
  required:
    - workbook_or_vba_text
    - user_goal
  optional:
    - error_log
    - expected_behavior

phases:
  P0_SPEC:
    output:
      - target_modules
      - user_goal
      - no_edit_confirmation

  P1_PLAN:
    output:
      - inspection_steps
      - risk_areas
      - plan_signature

  P2_TRACE:
    steps:
      - inventory_modules
      - identify_entry_points
      - trace_call_paths
      - locate likely change points
      - separate confirmed bugs from risks
```

```
  P3_AUDIT:
    output:
      - exact change locations
      - manual edit instructions
      - risks
      - test checklist
```

For the user, this produces a practical outcome. For the agent, it creates a stable execution path.

---

12. Why ENIAC/IAS Is a Useful Metaphor

The ENIAC/IAS distinction is historically meaningful as a metaphor for agent control.

ENIAC-mode represents fixed wiring:

```
Known procedure.
Known sequence.
No branching.
Strict trace.
```

IAS-mode represents stored-program control:

```
Program counter.
Memory.
Branching.
Flags.
Conditional execution.
```

AI agents need both.

A linear document-generation task may be ENIAC.
A debugging task is usually IAS.
A complex enterprise workflow may be a DAG built out of ENIAC and IAS subroutines.

The metaphor also helps programmers understand the design quickly. It translates fuzzy agent behaviour into familiar computational terms.

---

13. Limitations

This proposal does not make AI agents perfectly deterministic.

Several limitations remain:

1. Some tasks require creativity or judgment.
2. Some output states cannot be automatically verified.
3. Human confirmation is still needed for high-risk decisions.
4. LLM reasoning may still be wrong inside a bounded step.
5. Overly rigid protocols can reduce useful exploration.
6. Skill design itself requires expertise.
7. Validators can be incomplete or misconfigured.

8. Business tasks may contain hidden assumptions not captured in the input state.

Therefore, the goal is not full determinism.

The goal is:

> disciplined nondeterminism.

That means the model may still reason probabilistically, but the workflow around it is explicit, traceable, and bounded.

---

14. Implementation Roadmap

A practical implementation can proceed in five stages.

Stage 1: Strict Skill Builder

Create a skill that asks the user structured questions:

```
What is the goal?
What input state is required?
What output state proves success?
What files may be changed?
What files must not be changed?
What assumptions are forbidden?
What tests or validators should run?
What should happen on failure?
```

The output is a strict skill document.

Stage 2: Skill Compiler

Convert the strict skill document into YAML or JSON:

```
Markdown Skill → State Contract YAML
```

Stage 3: Step Runner

Run one step at a time:

```
current_state + step_instruction → LLM output → validator
```

Stage 4: Validator Library

Add reusable validators:

```
schema check
diff check
evidence check
test runner
lint runner
SQL parser
Excel structure validator
```

Stage 5: Enterprise Kernel

Integrate with:

```
GitHub Actions
CI/CD
Claude Code
Codex
OpenCode
VS Code
Jira
Slack
internal audit logs
```

---

15. New Professional Role: Agent Skill Engineer

The proposed role is not merely prompt engineer.

An Agent Skill Engineer must understand:

Area	Required Competence
Business analysis	Convert messy human tasks into process definitions
Software engineering	State, tests, diffs, rollback, version control
Prompt engineering	Guide LLM reasoning inside bounded steps
QA	Define validators and acceptance criteria
Governance	Audit trails, risk levels, permissions
Workflow design	Break work into reusable skills

This role is highly suitable for programmers because it reuses their existing mental models.

The labour-market implication is important:

> Programmers displaced from routine coding can become the people who make AI agents reliable enough for enterprise work.

---

16. Conclusion

AI agents are becoming powerful, but power without procedural discipline produces unstable automation. The immediate need is not only stronger models, but stricter execution structures.

ENIAC/IAS-style state-transition protocols offer a practical way to impose that structure. They transform agent skills from vague prompt instructions into auditable, reusable, state-governed workflows.

The key pattern is simple:

```
input state
→ operation
→ output state
→ verification gate
→ next state
```

ENIAC-mode handles fixed linear procedures. IAS-mode handles branching, debugging, recovery, and test-driven loops. Together, they provide a useful grammar for strict agent execution.

The “today” factor makes this especially urgent. Many programmers need a new economic role. Many businesses need reliable AI execution. Current agent platforms already provide enough primitives to build skills, hooks, validators, and execution kernels. The convergence of these

forces creates a significant opportunity: the emergence of Agent Skill Engineering as a practical discipline and Agent Reliability Infrastructure as a major software category.

The next phase of AI adoption will not be won only by more intelligent models. It will be won by systems that can make intelligent models work reliably.

In that sense, strict state-transition protocols may become one of the missing control layers of the agent economy.

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5, GLM's GLM-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.